Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

`https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones`

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128\times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32\times 128\times 9}.$$

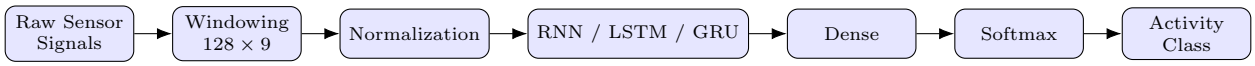
The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9.
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

$$70\% \text{ training}, \qquad 15\% \text{ validation}, \qquad 15\% \text{ testing}.$$

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

```
Input tensor shape:
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)
```

```
Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

The actual values of N_{train}, N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

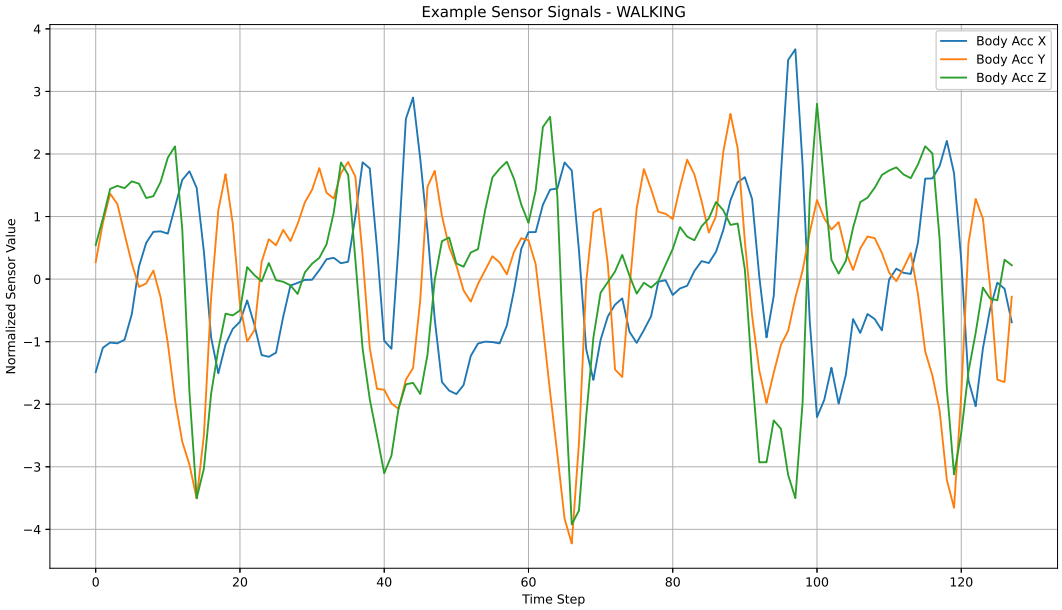


Figure 1: Sensor Signal Variation Across Time

Inference: Sensor signals vary across the 128 time steps,with movement based activities showing stronger temporal fluctuations than stationary activities. Ordering of measurements is important because recurrent models learn how sensor values change over time.

7. Vanilla RNN Architecture

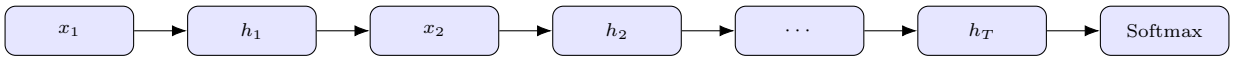
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

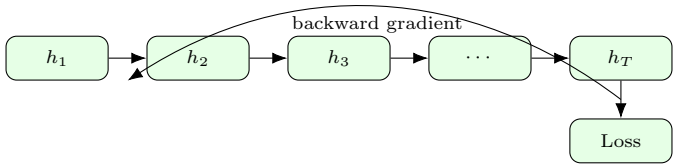
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left|\frac{\partial h_t}{\partial h_{t-k}}\right| \to 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \qquad x_2 = 0.7, \qquad x_3 = 0.2,$$

$$h_0 = 0, \qquad W_x = 0.5, \qquad W_h = 0.8, \qquad b = 0.1.$$

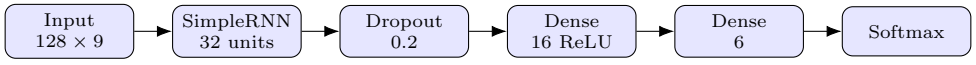
Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1, h_2 and h_3 manually.
Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.
The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

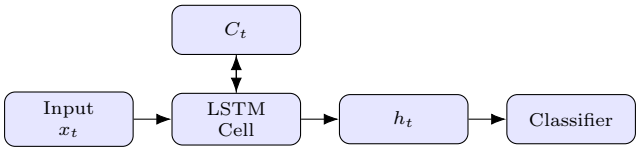
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

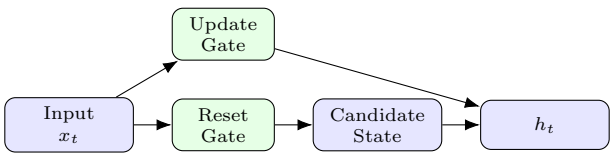
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

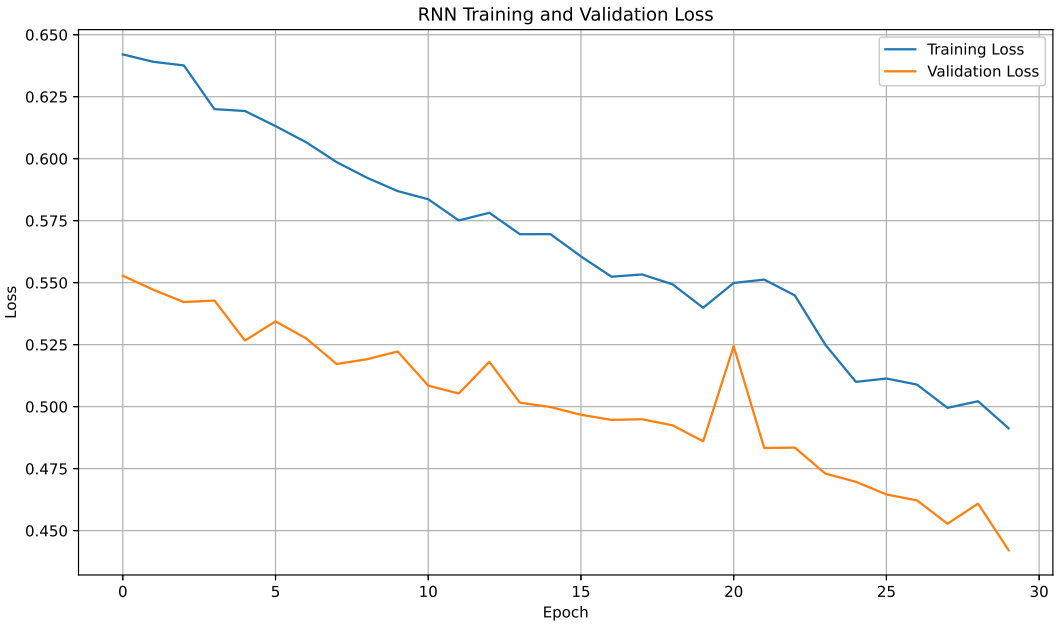


Figure 2: RNN loss curve

Inference: The RNN training loss decreases as the epochs progress, showing that the model learns the activity classification task.The validation loss generally follows the training trend, indcating reasonable convergence.

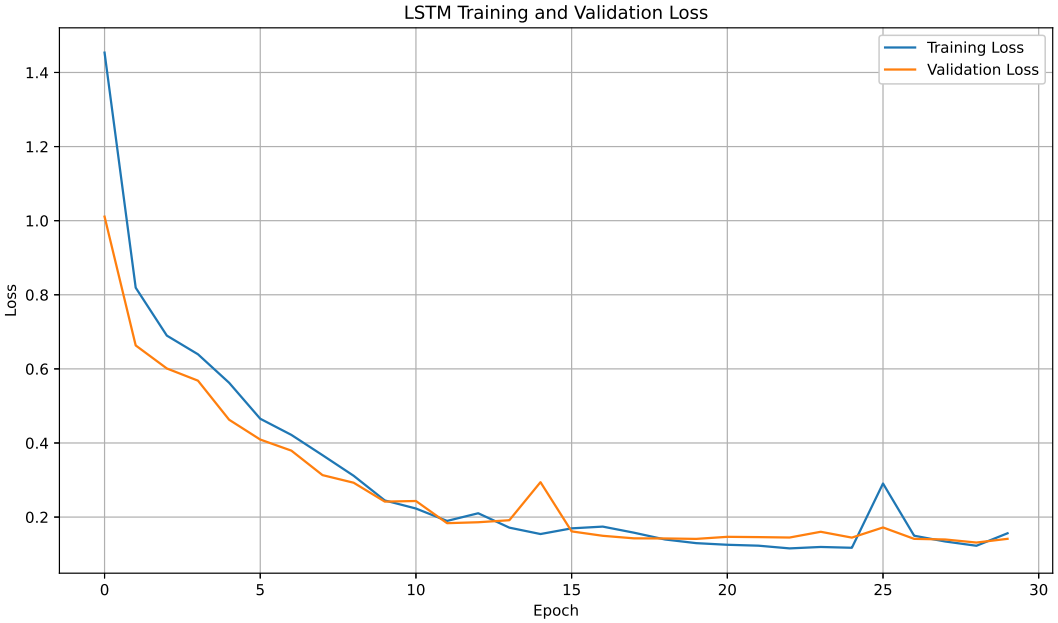


Figure 3: LSTM loss curve

Inference: The LSTM loss decreases steadily during training, indicating effective learning of temporal patterns. Its stable validation loss shows better handling of temporal dependencies.

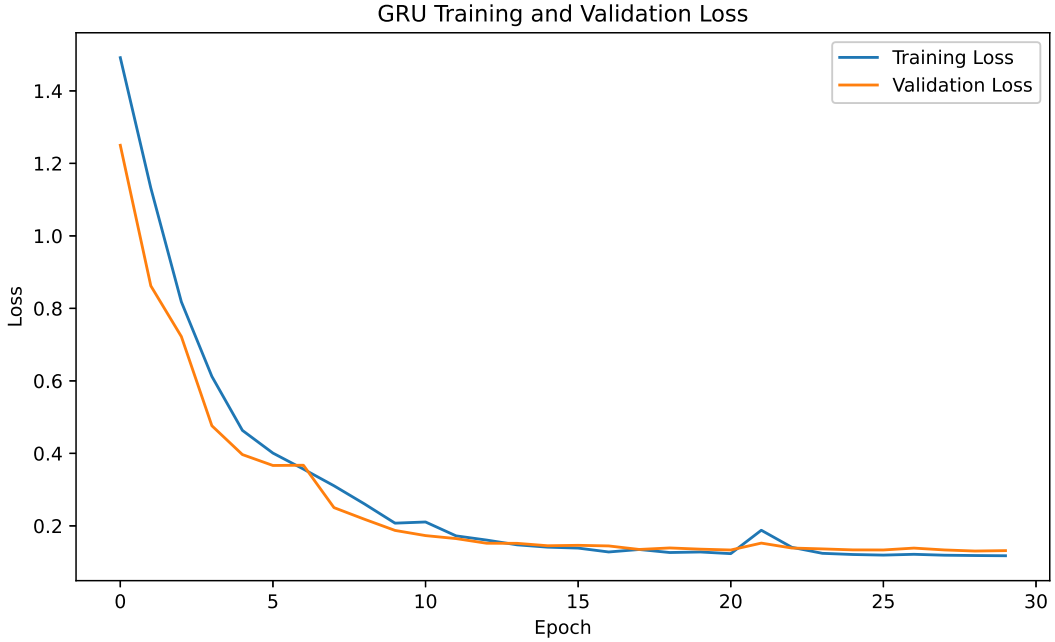


Figure 4: GRU loss curve

Inference: The GRU training loss decreases with increasing epochs and the validation loss follows the learning trend. This indicates that the GRU successfully learn temporal features from the sensor sequences.

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

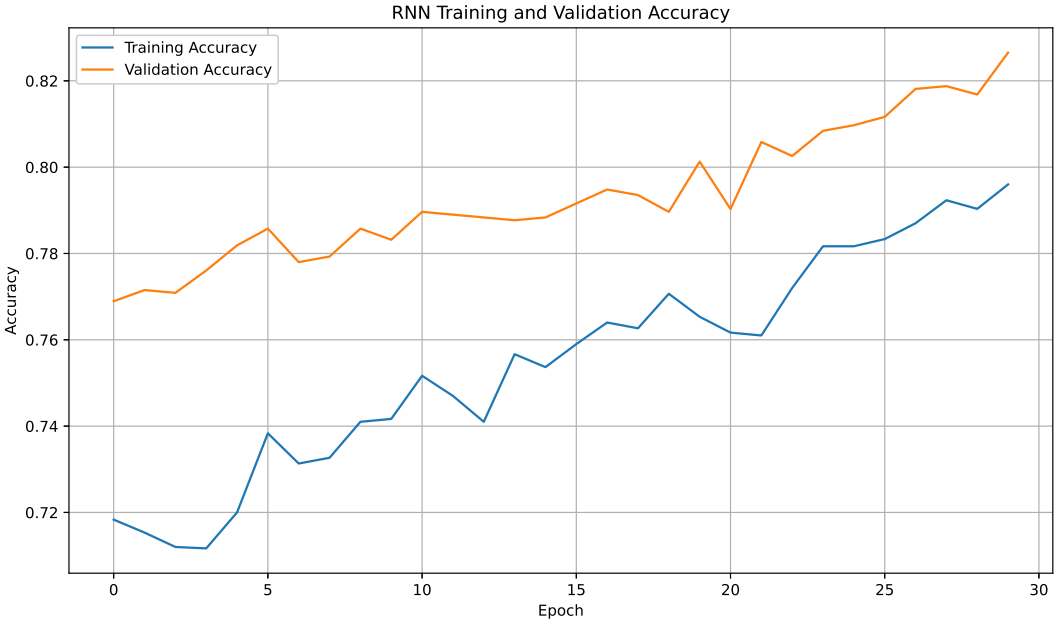


Figure 5: RNN accuracy curve

Inference: RNN accuracy increases during training and gradually converges. Its lower final validation performance indicate that the basic RNN has more difficulty capturing long-term temporal dependencies.

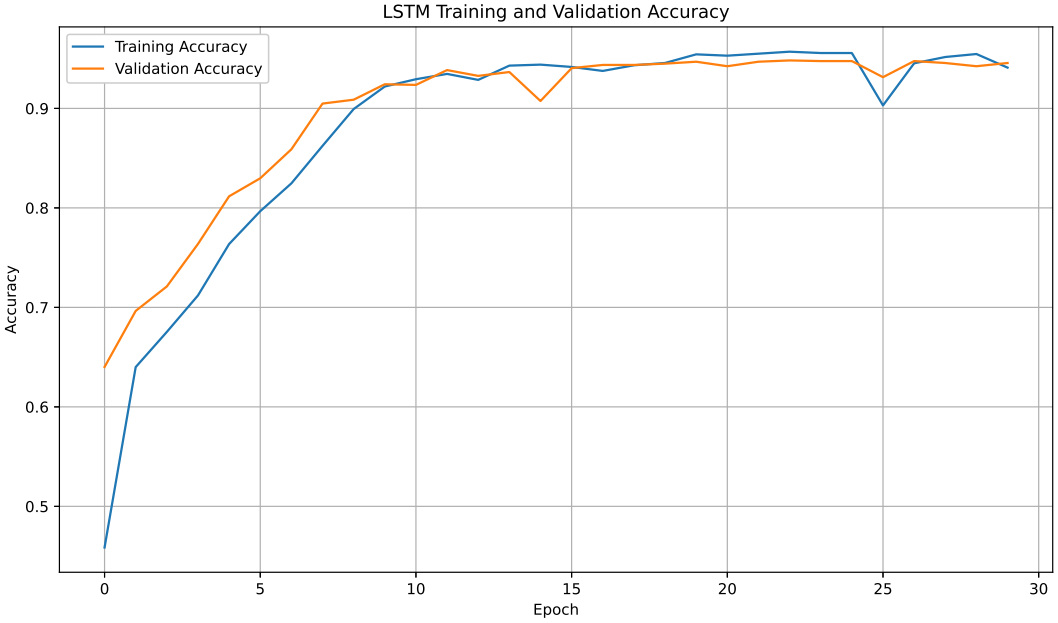


Figure 6: LSTM accuracy curve

Inference: LSTM accuracy increases steadily and reaches a high validation accuracy. The relatively small training-validation gap indicates good generalization on the activity classification task.

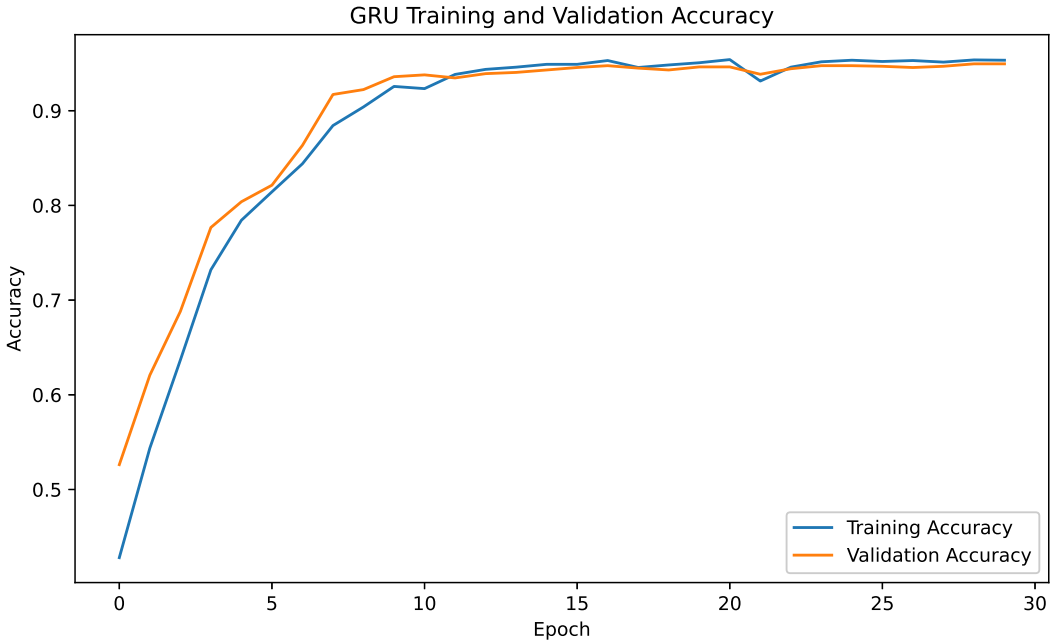


Figure 7: GRU accuracy curve

Inference: GRU accuracy improves throughout training and reaches high validation performance. The result indicates that the GRU effectively captures temporal information with its gated recurrent structure.

14. Performance Evaluation

Table 1: Performance Comparison of RNN, LSTM and GRU

Metric	**RNN**	**LSTM**	**GRU**
Accuracy (%)	79.87	92.82	94.17
Macro Precision (%)	78.89	93.20	94.52
Macro Recall (%)	78.97	93.24	94.47
Macro F1 (%)	78.91	93.17	94.49
Parameters	1,974	6,006	4,758
Training Time (s)	69.77	133.65	211.60

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

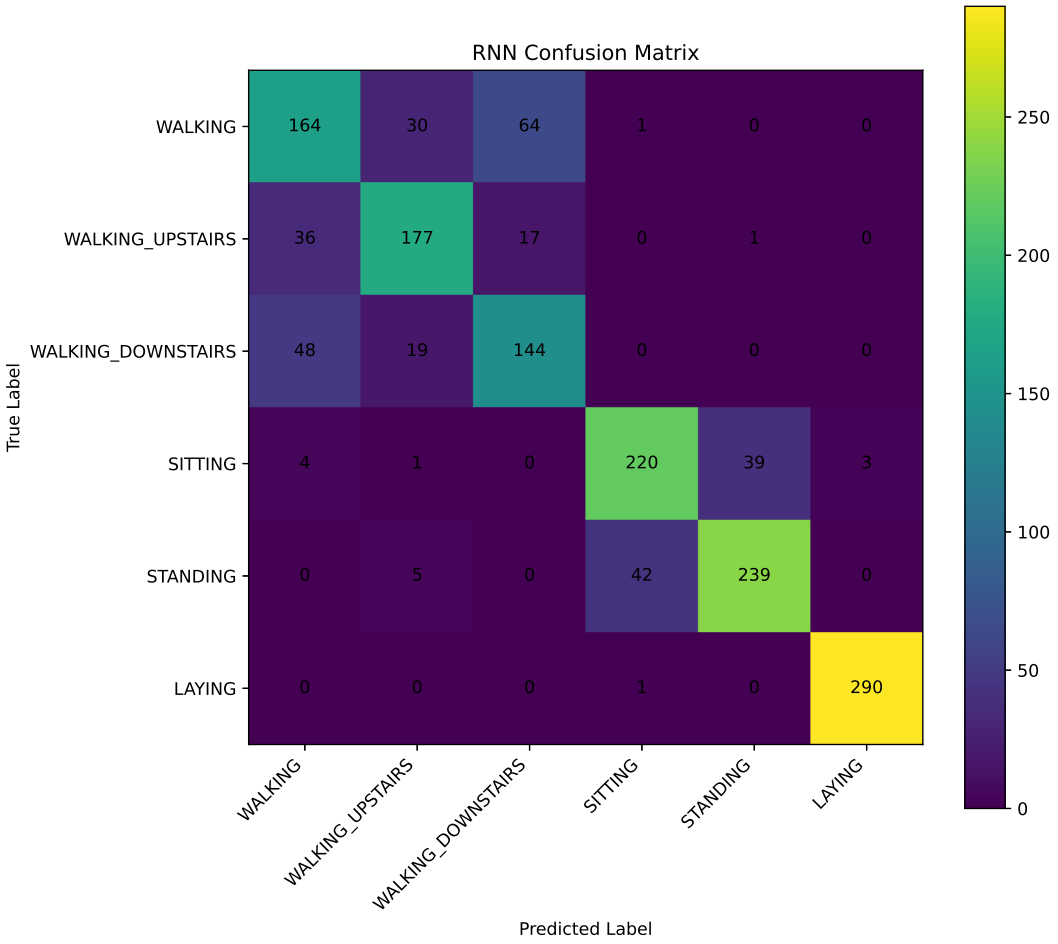


Figure 8: RNN confusion matrix

Inference: Most samples are classified along the diagonal, while some errors occur between activities with similar motion patterns. The confusion matrix shows the class-wise errors that cannot be observed from overall accuracy alone.

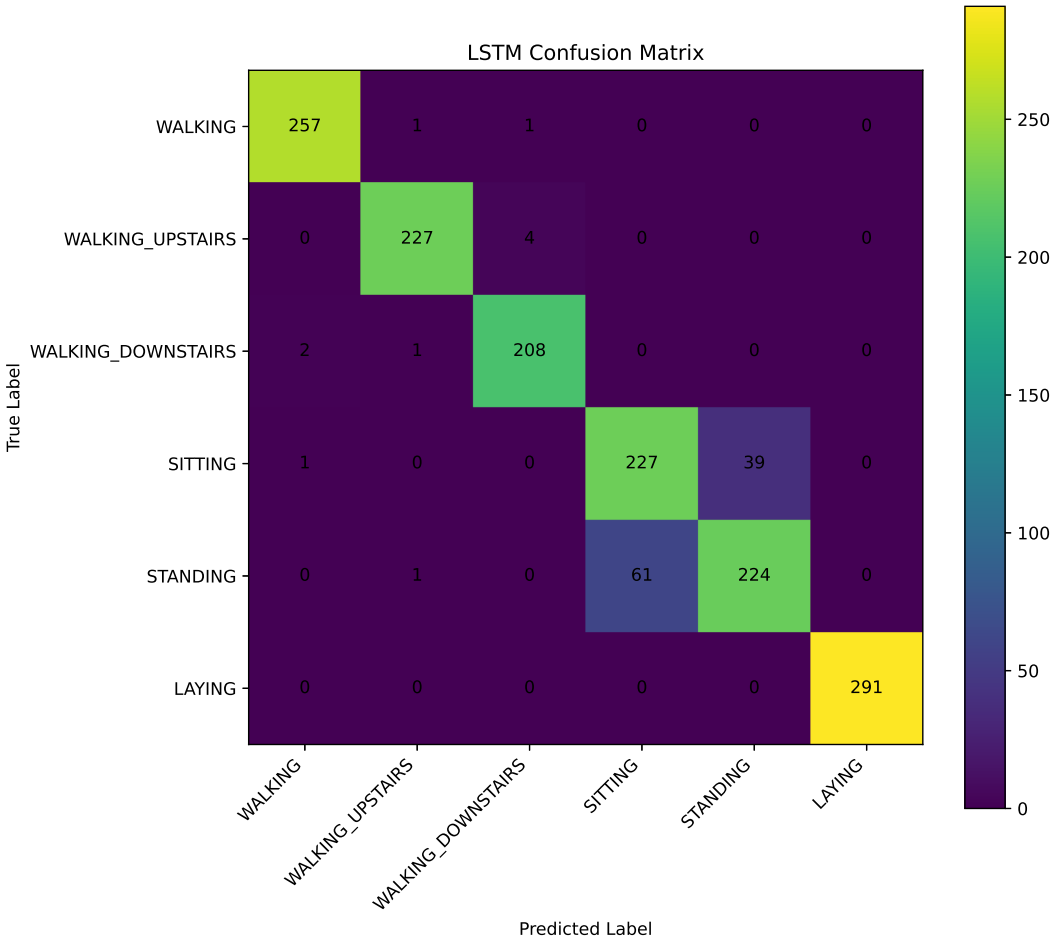


Figure 9: LSTM confusion matrix

Inference: The LSTM confusion matrix shows a strong concentration of predictions along the diagonal, indicating good recognition across the six activities. Remaining errors mainly occur between activities with similar temporal characteristics.

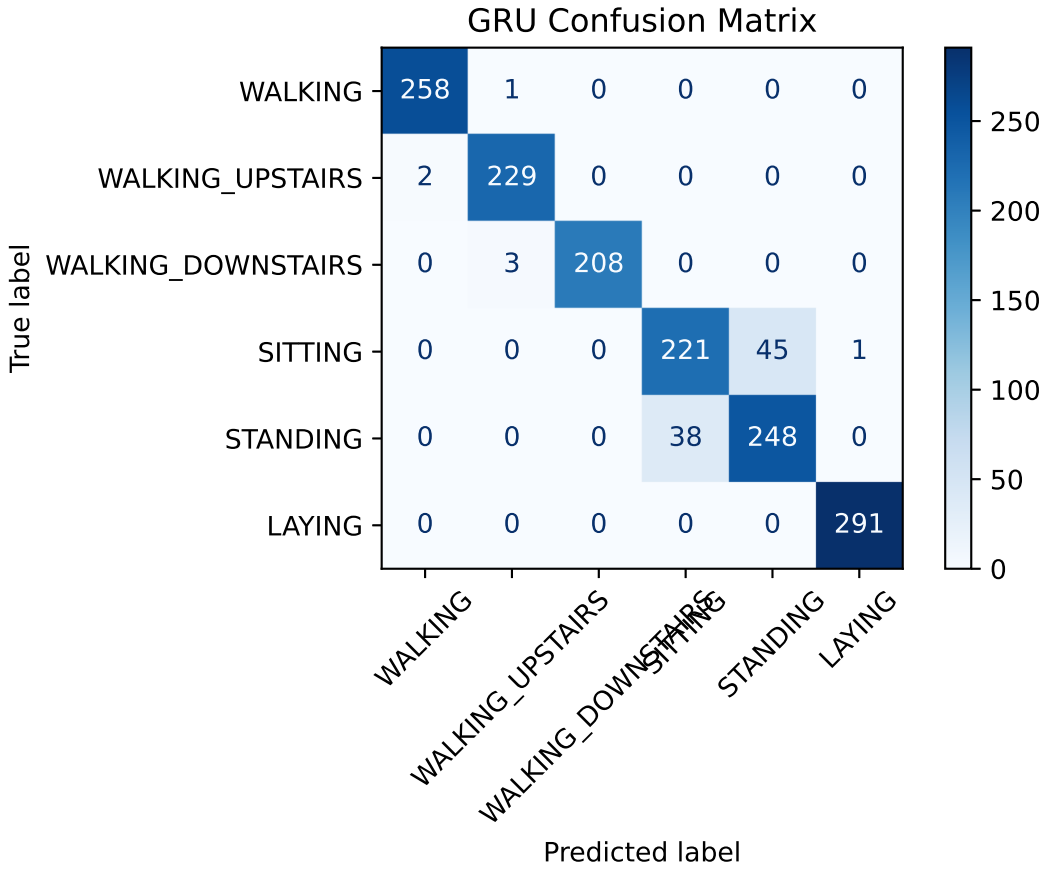


Figure 10: GRU confusion matrix

Inference: The GRU produces a strongly diagonal confusion matrix, indicating high class recognition. Misclassifications are mainly associated with activities having similar sensor-motion patterns.

16. RNN vs. LSTM vs. GRU Comparison

Table 2: Comparison of RNN, LSTM and GRU Properties

Property	**RNN**	**LSTM**	**GRU**
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	69.77	133.65	211.60
Test F1-score (%)	78.91	93.17	94.49

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

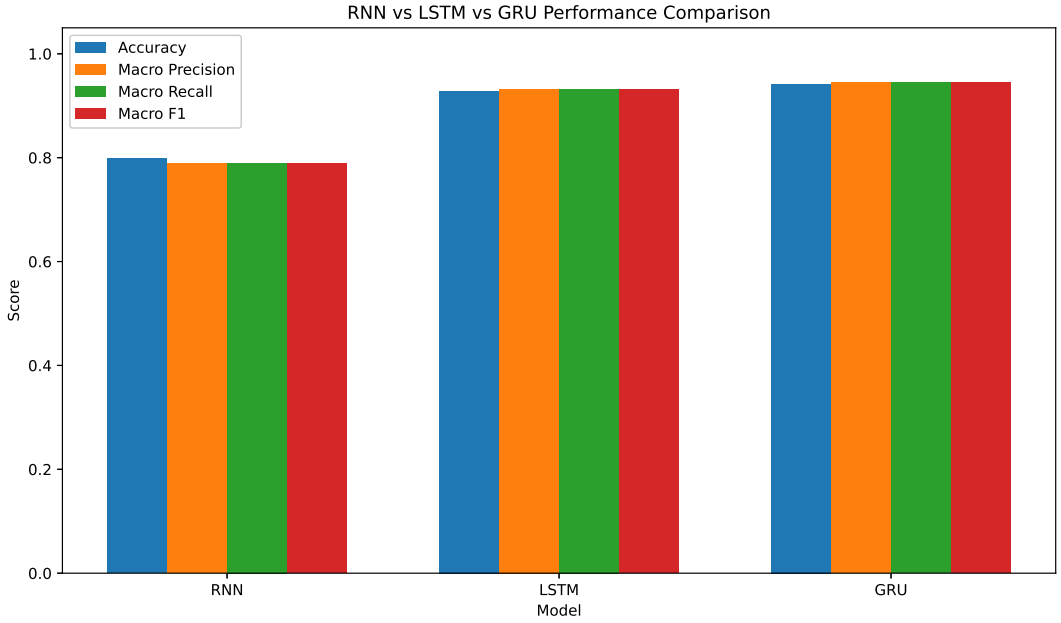


Figure 11: Performance Comparison

Required inference:
The three models show different predictive performance, with LSTM and GRU achieving substantially higher test performance than the vanilla RNN. The results demonstrate the benefit of gated recurrent architectures for learning temporal dependencies.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	75.09	91.08	91.36
64	70.16	86.23	93.90
128	69.69	69.95	95.02

Plot 6: Sequence Length vs. Test F1-score

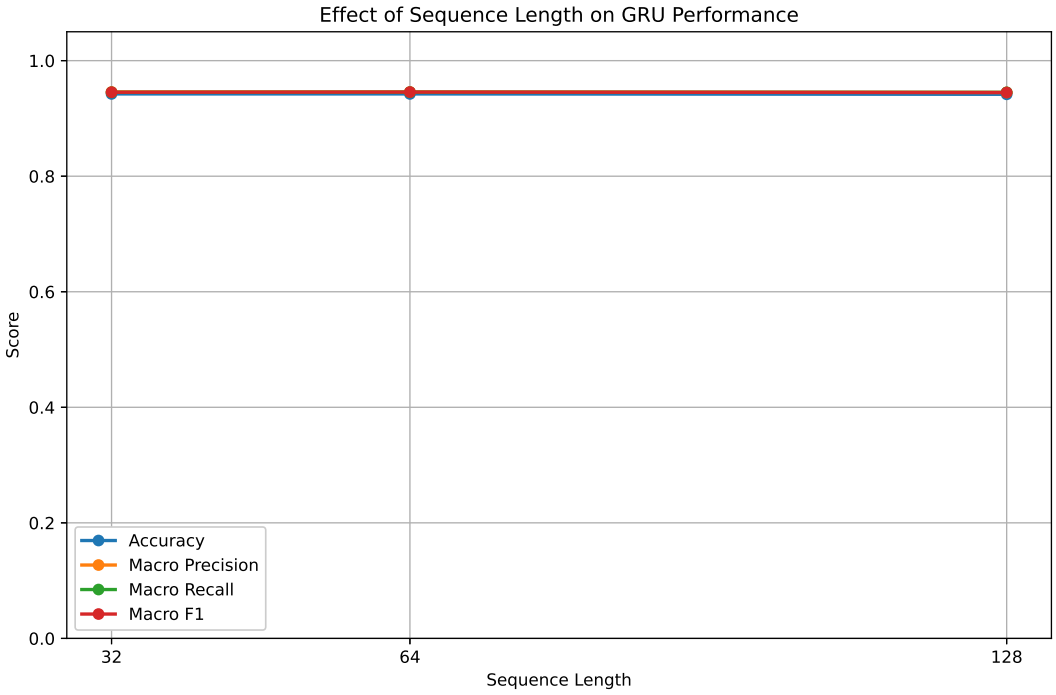


Figure 12: sequence length performance

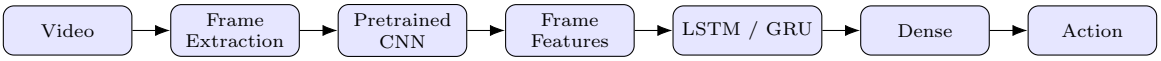
Inference: Changing the sequence length changes the amount of temporal information available to the recurrent model. Longer sequences provide more context but can also increase computational cost and training requirements.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.
The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:
`https://www.crcv.ucf.edu/data/UCF101.php`
Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.
Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.
If the CNN produces a feature vector of dimension D, then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.
For example:

```
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
```

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.

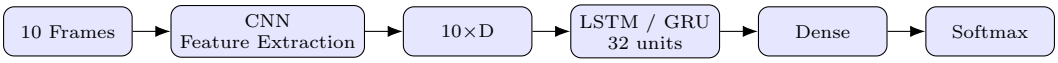


Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:
Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames
Display a representative sequence of sampled frames from one video.

Inference : The sampled frames show the spatial content of the video at different time steps. The CNN extracts spatial features from individual frames, while the recurrent network uses the ordered feature sequence to model temporal changes.

Plot 8: Video Training and Validation Curves

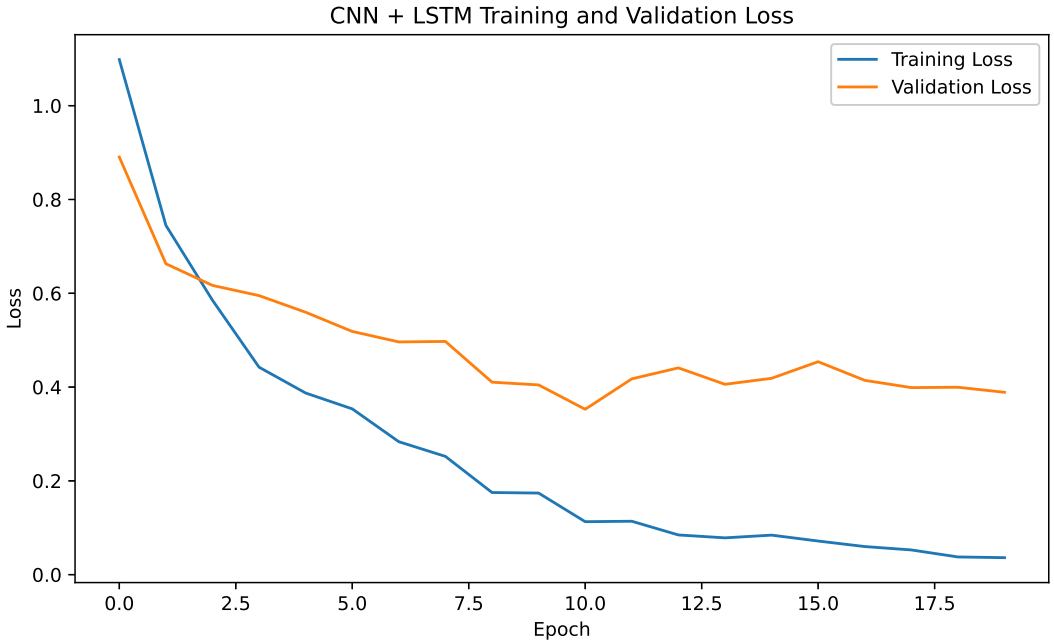


Figure 13: Loss curve

Inference : The CNN-LSTM training loss decreases as training progresses, indicating that the recurrent classifier learns from the extracted frame features. The validation trend indicates how well the learned temporal representation generalizes to unseen videos.

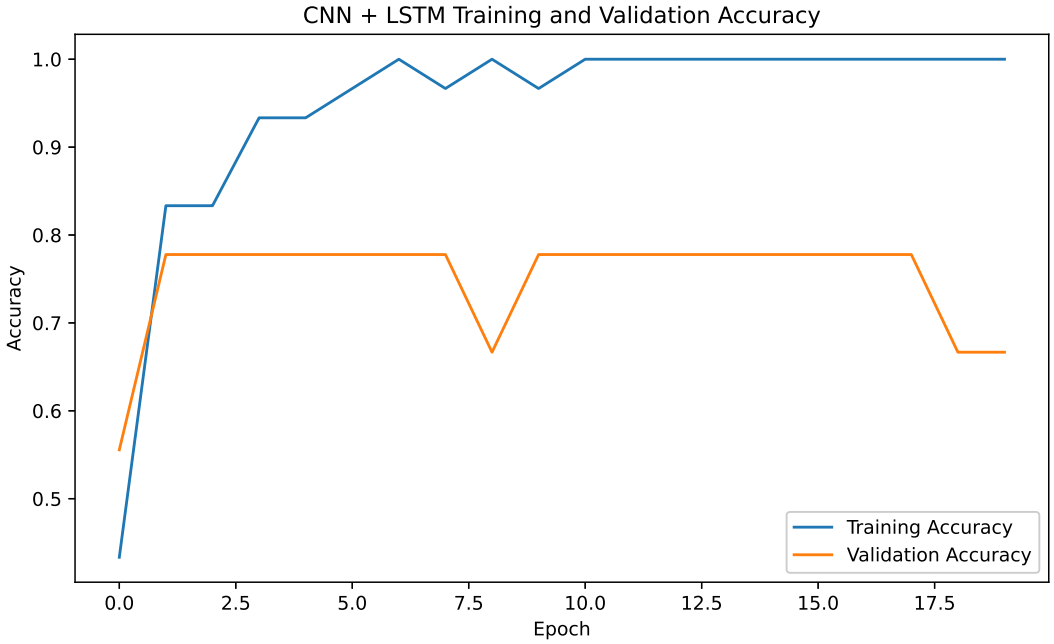


Figure 14: accuracy curve

Inference : The CNN-LSTM accuracy increases during training, showing improved recognition of the selected video classes. The validation accuracy indicates the model's ability to generalize temporal patterns beyond the training videos.

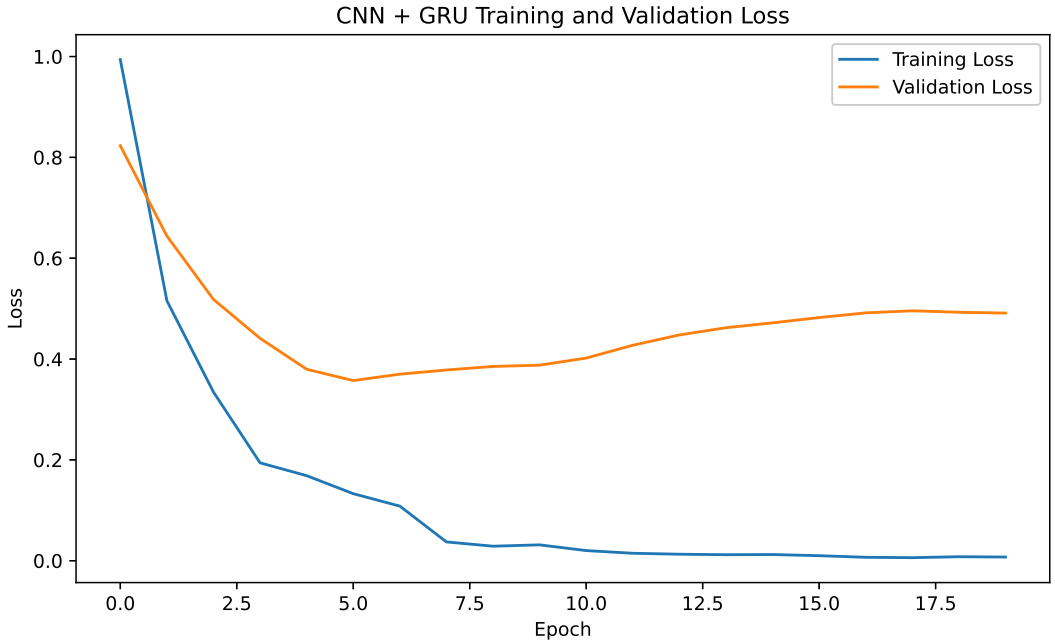


Figure 15: accuracy curve

Inference : The CNN-GRU loss decreases during training, demonstrating learning from the CNN-extracted feature sequences. The validation loss provides an indication of its generalization to unseen video samples.

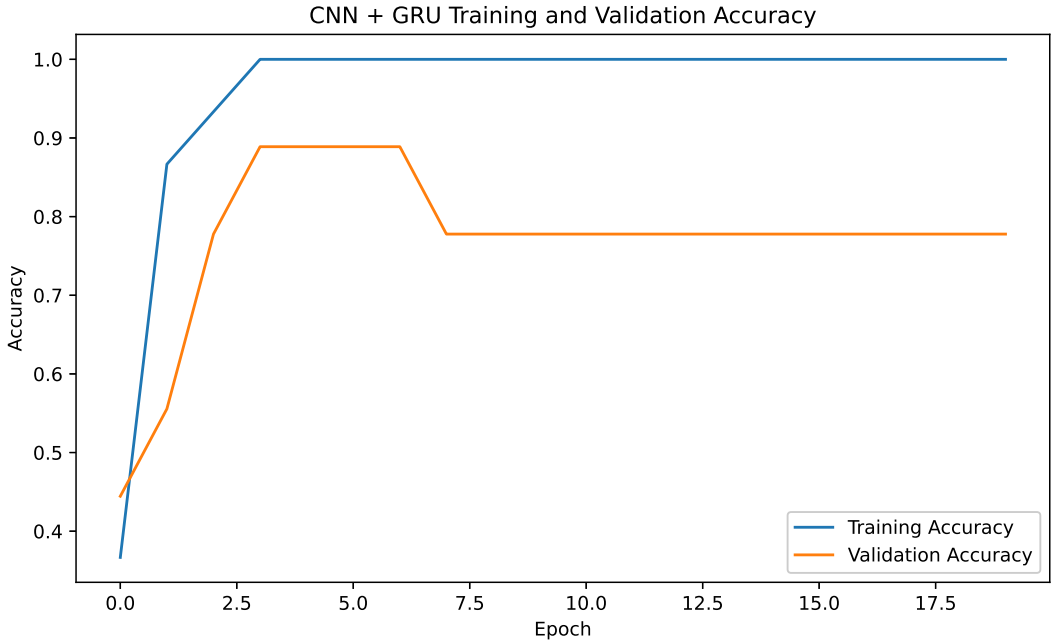


Figure 16: accuracy curve

Inference : The CNN-GRU accuracy improves over the training epochs as temporal relationships between frame features are learned. Its validation accuracy reflects recognition performance on unseen videos.

Plot 9: Video Confusion Matrix

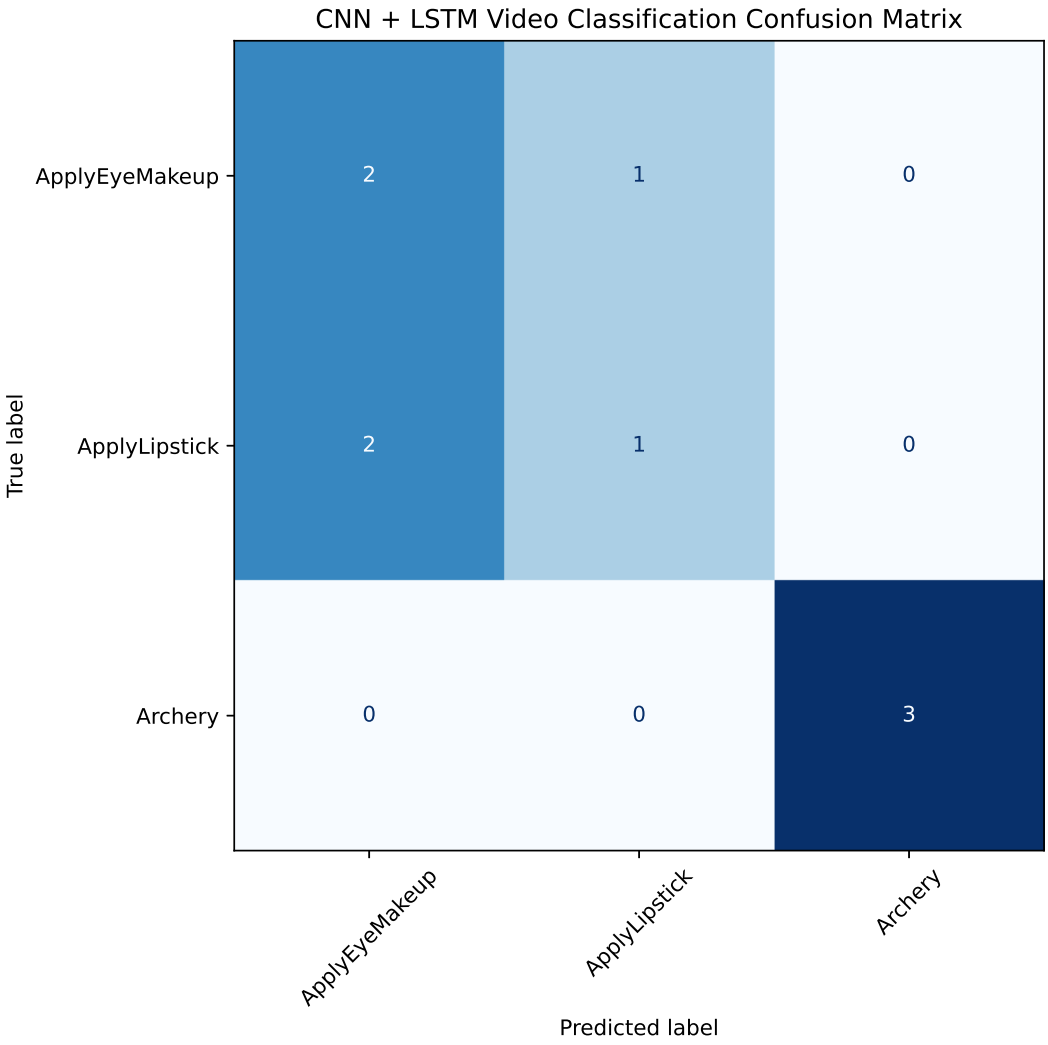


Figure 17: LSTM confusion matrix

Inference : The confusion matrix shows how effectively the CNN-LSTM distinguishes the selected video classes. Strong diagonal values indicate correct recognition, while off-diagonal values represent confusions between visually or temporally similar actions.

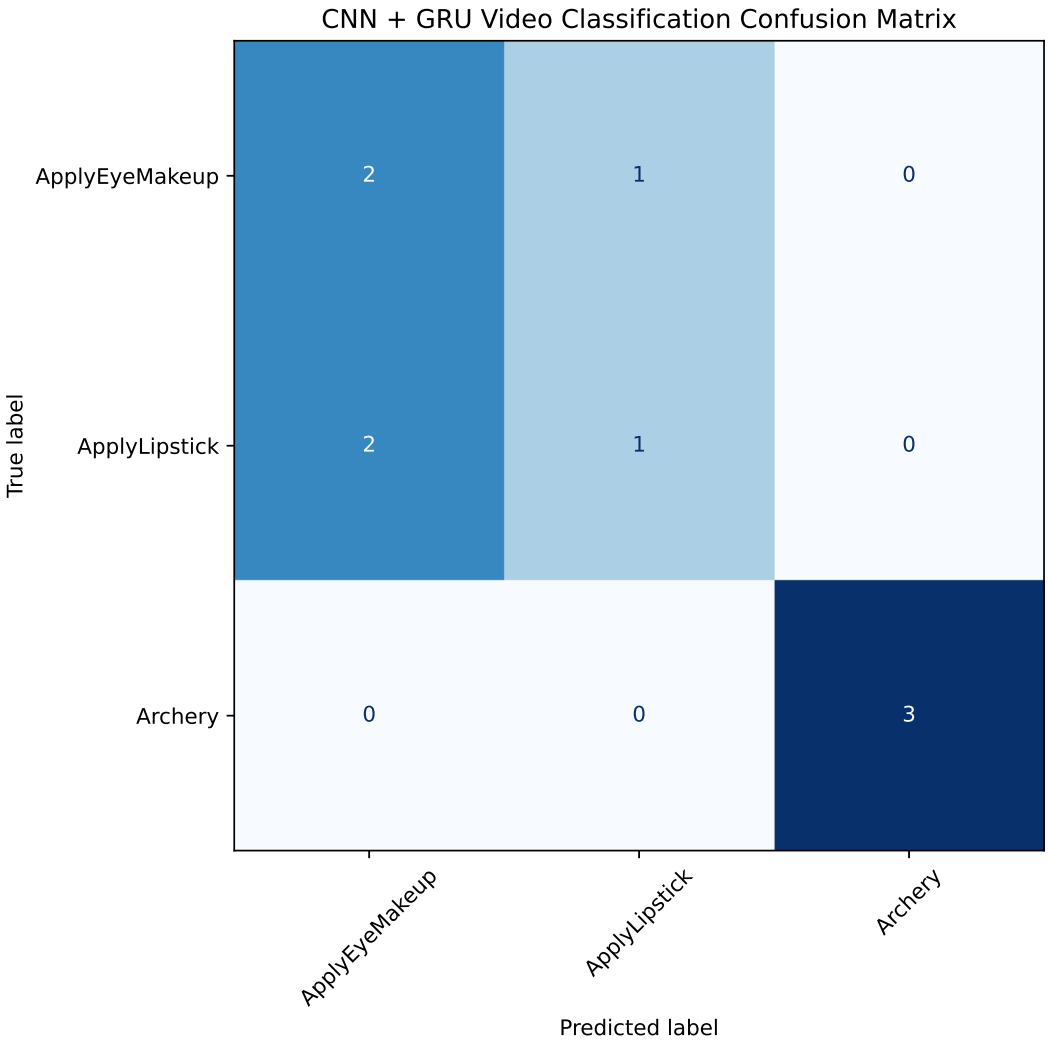


Figure 18: GRU confusion matrix

Inference : The CNN-GRU confusion matrix shows the class-wise recognition of the selected actions. Errors in off-diagonal cells indicate cases where the temporal patterns of different actions are difficult to distinguish.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder–decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

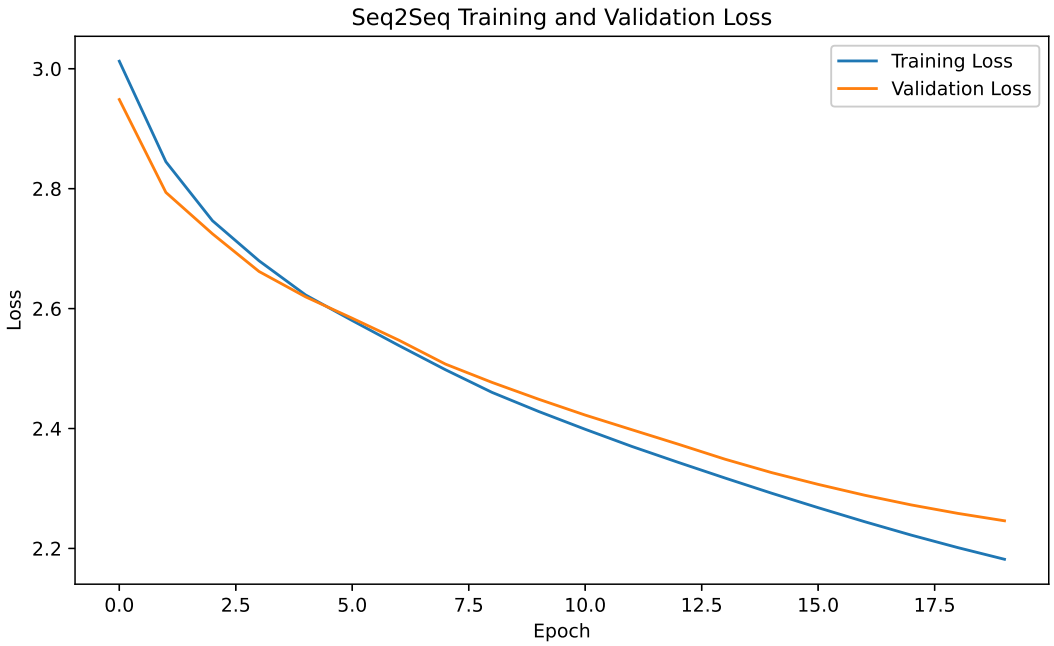


Figure 19: Loss Curve

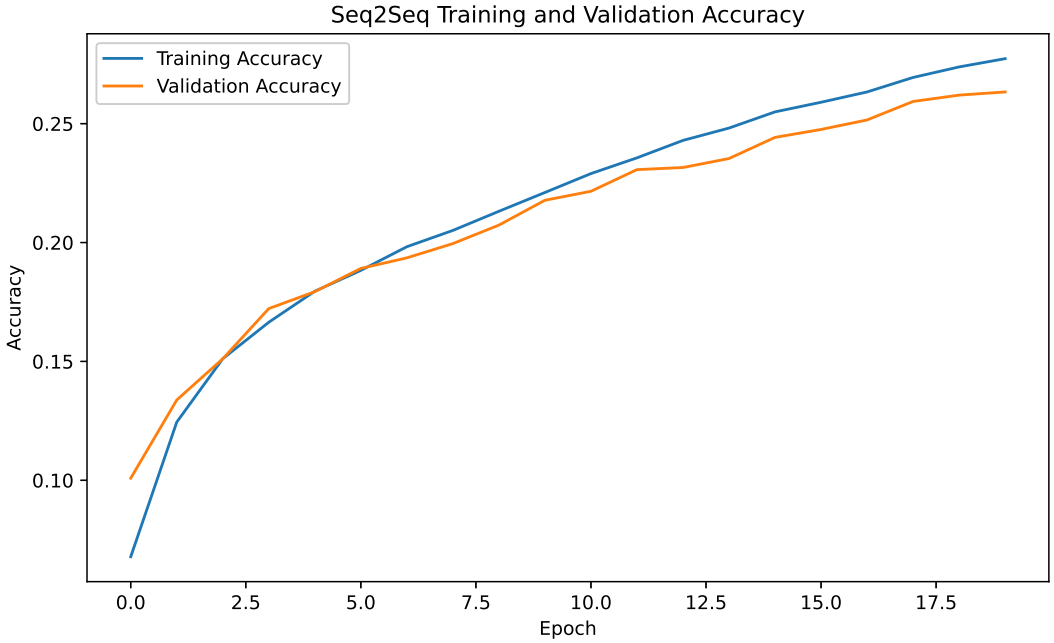


Figure 20: Accuracy Curve

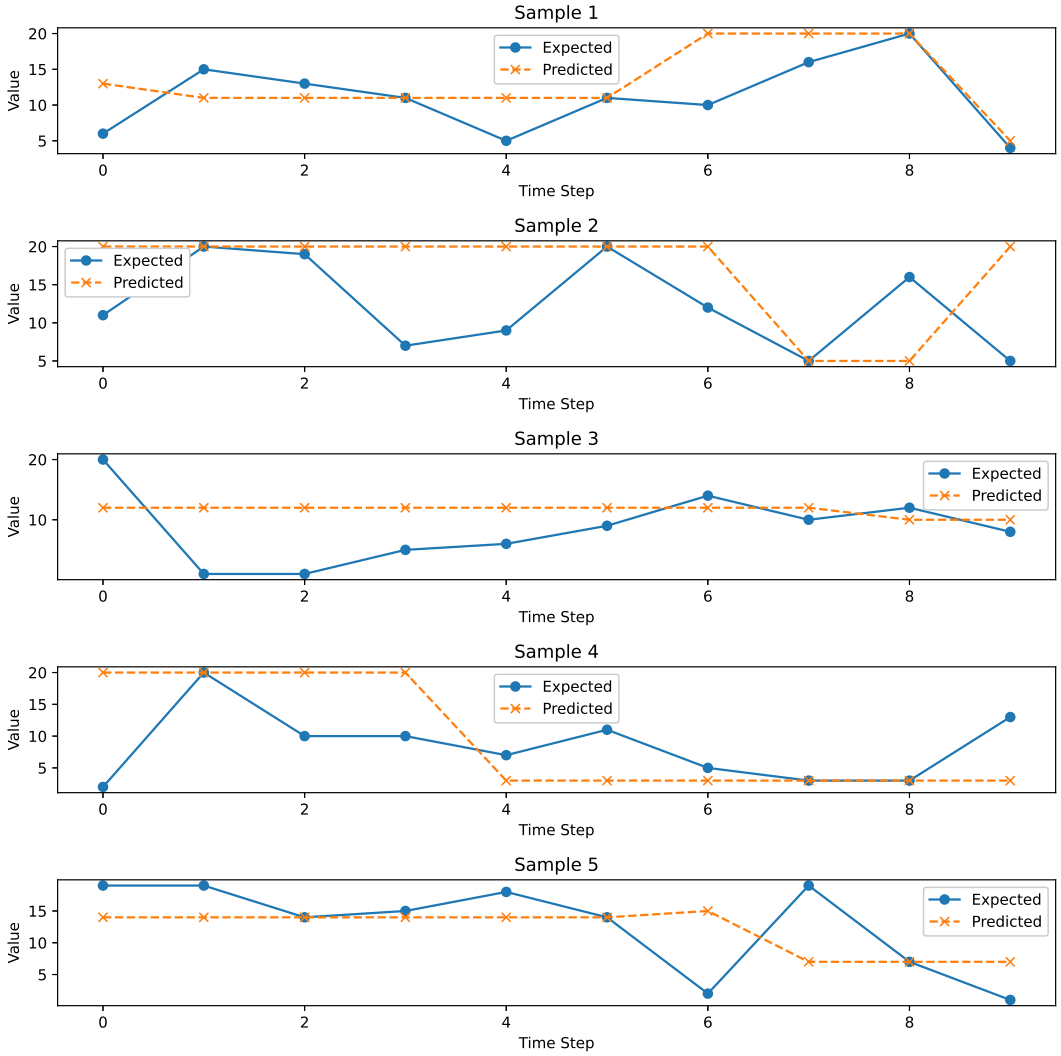


Figure 21: Predictions Curve

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

25. Consolidated Results

Table 3: Consolidated Results

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	79.87	78.89	78.97	78.91	1,974
LSTM	92.82	93.20	93.24	93.17	6,006
GRU	94.17	94.52	94.47	94.49	4,758
CNN–LSTM/GRU	0.666667 / 0.666667	–	–	0.657143 / 0.657143	168643 / 126723

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

1. What is a Recurrent Neural Network (RNN)?
   Ans: RNN is a neural network designed to process sequential data. It uses information from previous time step to process the current input.
2. What information is represented by the hidden state?
   Ans: The hidden state stores information about previous inputs in the sequence. It acts as the networks memory.
3. Why is sequence order important?
   Ans: The order of elements can change the meaning of a sequence.RNNs consider the order in which the inputs occur.
4. Explain the difference between a feed-forward network and an RNN.
   Ans: A feed-forward network processes each input independently, while an RNN maintains a hidden state to remember information from previous inputs.
5. Explain Backpropagation Through Time (BPTT).
   Ans: BPTT is the method used to train RNNs by unfolding RNN across time steps and propagating the error backward through those steps.
6. Why can gradients vanish during BPTT?
   Ans: When gradients are repeatedly multiplied by small values during backpropagation, they become very small.This makes learning long-term dependencies difficult.
7. Why can gradients explode during BPTT?
   Ans: When gradients are repeatedly multiplied by large values,they can become extremely large. This causes unstable training.
8. What are long-term dependencies?
   Ans: Long-term dependencies occur when information from an earlier part of a sequence is needed to understand or predict a later part.
9. What is the role of the LSTM cell state?
   Ans: The LSTM cell state acts as a long-term memory path that carries important information across time steps.
10. What are the forget, input and output gates in LSTM?

Ans:
- **Forget gate:** Decides what information to remove.
- **Input gate:** Decides what new information to store.
- **Output gate:** Decides what information to output as the hidden state.

11. What are the reset and update gates in GRU?
Ans:
- **Reset gate:** Controls how much previous information is ignored.
- **Update gate:** Controls how much old information is retained and how much new information is added.

12. Compare LSTM and GRU.
Ans: LSTM uses three gates and a separate cell state, while GRU uses two gates and combines memory and hidden state.So GRU has fewer parameters.
13. Why does GRU generally use a simpler gating mechanism than LSTM?
Ans: GRU combines some of LSTMs functions into fewer gates and does not use a separate cell state. This makes its architecture simpler.
14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
Ans: Using the same settings makes the comparison fair. Differences in performance can then be attribted mainly to the recurrent architecture.
15. What does the confusion matrix reveal that accuracy alone does not?
Ans: A confusion matrix shows which classes are correctly classified and which classes are confused with each other.
16. Why should macro F1-score be reported for multi-class classification?
Ans: Macro F1 calculates the F1-score for each class and then gives them equal importance. It shows performance across all classes.
17. What is the significance of sequence length?
Ans: Sequence length determines how many time steps are given to the model. Different lengths can affect the amount of temporal information available and the computational cost.
18. Why can a CNN be used as a feature extractor for video?
Ans: A CNN can extract useful spatial features from individual video frames without requiring the CNN to be trained from scratch.
19. What type of information is extracted by the CNN?
Ans: The CNN extracts spatial information such as shapes,edges,textures and objects present in each frame.
20. What type of information is learned by the recurrent network?
Ans: The recurrent network learns temporal information, such as how features change across frames.
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
Ans: Each frame produces a feature vector. Arranging these vectors in time order allows the LSTM/GRU to learn the temporal relationships between frames.
22. Explain the encoder and decoder in sequence-to-sequence learning.
Ans:
- Encoder: Reads the input sequence and creates a representation of it.
- Decoder: Uses this representation to generate the output sequence.

23. What is teacher forcing?
Ans: Teacher forcing is a training technique where the decoder receives the actual previous output instead of its own predicted output.
24. Differentiate token accuracy and sequence accuracy.
Ans:
- Token accuracy: Measures how many individual tokens are predicted correctly.
- Sequence accuracy: Counts a sequence as correct only when all its tokens are correct.

25. Why must the test set remain untouched during model selection?
Ans: The test set should only be used for final evaluation. Using it during model selection can cause information from the test set to influence the model and give an unreliable performance estimate.

28. Additional Exercises

The following additional exercises were specified in the experiment:

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
5. Change the sequence length and study the effect on computational cost.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.

7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

CNN–LSTM vs. CNN–GRU

Performance Comparison

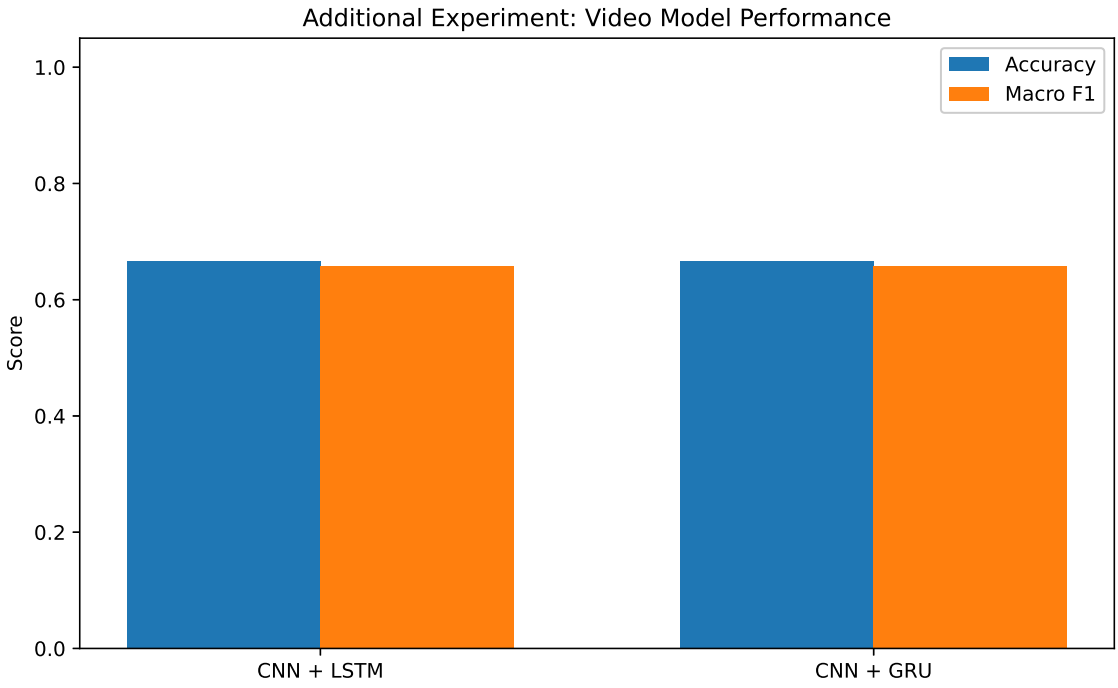


Figure 22: Performance comparison of CNN–LSTM and CNN–GRU

Inference:
The performance comparison shows the difference in predictive results between the CNN–LSTM and CNN–GRU models when both use the same CNN-extracted features.Since input features are identical, the observed difference is mainly associated with the recurrent architecture used for temporal modelling.

Efficiency Comparison

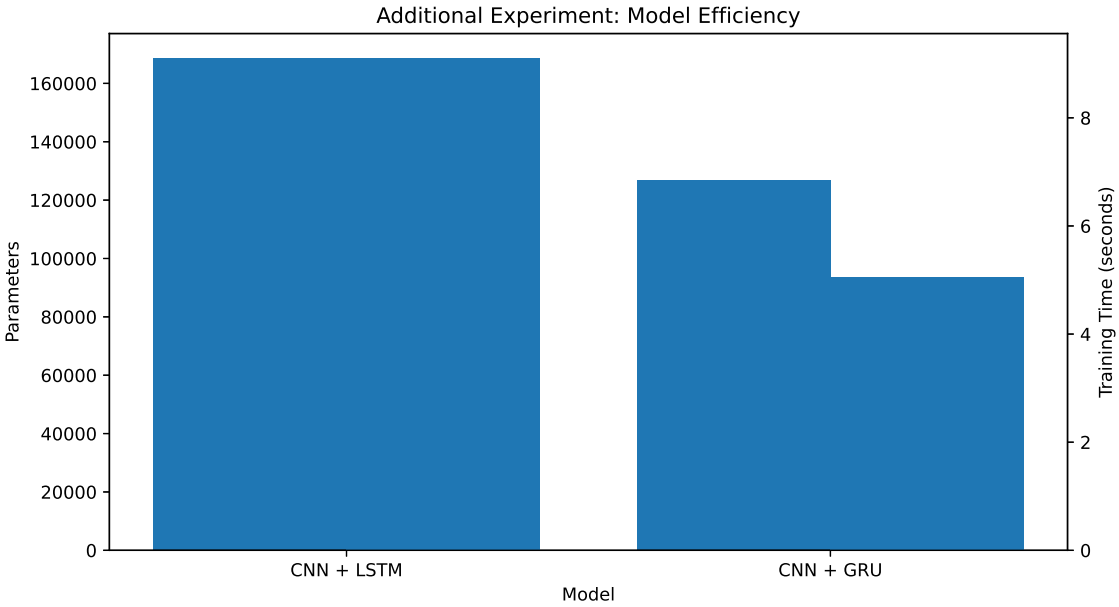


Figure 23: Efficiency comparison of CNN–LSTM and CNN–GRU

Inference:
The efficiency comparison shows the difference in computational requirements between the CNN-LSTM and CNN-GRU models. Because both models use the same CNN features,observed difference in parameters or computational cost is primarily due to the recurrent layers.

Overall Observation

This additional experiment demonstrates that the recurrent architecture can affect both predictive performance and computational requirements even when the extracted CNN features remain unchanged. The experiment therefore provides a controlled comparison between LSTM and GRU for temporal modelling of video features.

Source Code

GitHub Repository:
`https://github.com/Teena-Pranava/Deep-Learning-Lab`

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

$$\boxed{\text{Sequential Data} \rightarrow \text{Preprocessing} \rightarrow \text{RNN/LSTM/GRU} \rightarrow \text{Evaluation}}$$

and the video-understanding pipeline:

$$\boxed{\text{Video} \rightarrow \text{Frames} \rightarrow \text{CNN Features} \rightarrow \text{LSTM/GRU} \rightarrow \text{Action Prediction}}$$

They should also understand the encoder–decoder formulation:

$$\boxed{\text{Input Sequence} \rightarrow \text{Encoder} \rightarrow \text{Context} \rightarrow \text{Decoder} \rightarrow \text{Output Sequence}}$$

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: `https://www.tensorflow.org`
7. Keras Documentation: `https://keras.io`